
Ağ Tabalı Paralel Dağıtım Sistemleri Proje Dokümanı

Proje - 1 ve Proje - 2



BLM4522 - Ağ Tabanlı Paralel Dağıtım Sistemleri

Github => <https://github.com/semihcngz/AgTabaliParalelDagitimSistemleri.git>

Semih Cengiz - 20290233

İçindekiler

İçindekiler	2
Giriş	3
Veritabanı Performans Optimizasyonu ve İzleme (Proje - 1)	4
Başlangıç Durumunun Analizi	4
Veritabanı izleme	6
Modül Ayarları	6
Eklenti Oluşturma	7
Aktif Sorguları İzleme	7
En Çok Zaman Harcayan Sorguları Tespit Etme	8
Yavaş Sorguları Loglama	9
İndeks Yönetimi	10
Mevcut İndeksleri Görüntüleme	10
İndeks Kullanım İstatistikleri	10
İndeks Eklenecek Sorgu	11
Index Eklendikten Sonra Performans	12
Sorgu İyileştirme	13
Sorgu Farkları	13
Veri Yönetici Roller	14
Erişim Yetkisi	15
Tablo Bazında Yetki	15
Verilen Yetkileri Listeleme	16
Verilen Yetkilerin Geri Alınması	17
Veritabanı Yedekleme ve Felaketten Kurtarma Planı (Proje - 2)	18
Tam, Artık, Fark Yedeklemeleri	18
Tam Yedekleme	18
Artık ve Fark Yedekleme	22
Zamanlayıcılarla Yedekleme	24
Yedekleme Aşamaları	24
Felaketten Kurtarma Senaryoları	25
Restore Aşamaları	25
Test Yedekleme Senaryoları	27

Giriş

Bu dokümanda iki farklı projenin tanıtımı yer almaktadır. İlk projede veri optimizasyonu ve izleme tarafına, ikinci projede ise yedekleme ve kurtarma taraflarına odaklanmıştır.

Her iki projede de PostgreSQL için özel olarak hazırlanmış Pagila veritabanı kullanılmıştır. Pagila DVD kiralama sistemine benzer bir yapıya ve tablolar sahiptir. Örneğin customer, film, rental, payment gibi tablolar içermektedir.

Veritabanı Performans Optimizasyonu ve İzleme

(Proje - 1)

Bu projede örnek veritabanı kullanarak PostgreSQL ile performans analizi ve optimizasyon çalışmaları yapılmıştır. Çalışmanın amacı; sorgu performansını izleme, mevcut indeks yapısını kontrol etme, uzun süren sorguları analiz etme gibi konular içermektedir. Ayrıca disk alanı kullanımı ve veri yöneticisi rolleri gibi başlıklar incelenmiştir.

Başlangıç Durumunun Analizi

table_name	row_count
film	1000
actor	200
customer	599
inventory	4581
rental	16044
payment	16044

Tabloların Satır Sayısı

Yukarıda gösterilen tablo veritabanının başlangıçta içerdiği tabloların büyüklüğünü göstermektedir. En yoğun tabloların rental ve payment olduğu görülüyor.

schemaname	table_name	total_size	total_size_bytes
public	rental	1920 kB	1966080
public	film	752 kB	770048
public	payment_p2007_03	512 kB	524288
public	film_actor	496 kB	507904
public	inventory	464 kB	475136
public	payment_p2007_04	440 kB	450560
public	payment_p2007_02	408 kB	417792

En Çok Yer Kaplayan Tablolar

Yukarıda tabloda tabloların diskte ne kadar yer kapladığı görülmektedir. Burada en fazla yer kaplayan rental tablosu olduğundan performans testlerinde özellikle kiralama işlemleriyle ilgili tabloların dikkate alınması gerektiğini göstermektedir. Ayrıca payment tablolarının parçalanmış halde bulunduğu görülmektedir.

schemaname	tablename	indexname	indexdef
public	actor	actor_pkey_incl	CREATE UNIQUE INDEX actor_pkey_incl ON public.actor USING btree (actor_id) INCLUDE (first_name, last_name)
public	actor	idx_actor_last_name	CREATE INDEX idx_actor_last_name ON public.actor USING btree (last_name)
public	film	film_fulltext_idx	CREATE INDEX film_fulltext_idx ON public.film USING gist (fulltext)
public	film	film_pkey	CREATE UNIQUE INDEX film_pkey ON public.film USING btree (film_id)
public	film	idx_fk_language_id	CREATE INDEX idx_fk_language_id ON public.film USING btree (language_id)
public	film	idx_fk_original_language_id	CREATE INDEX idx_fk_original_language_id ON public.film USING btree (original_language_id)
public	film	idx_title	CREATE INDEX idx_title ON public.film USING btree (title)
public	customer	customer_pkey	CREATE UNIQUE INDEX customer_pkey ON public.customer USING btree (customer_id)
public	customer	idx_fk_address_id	CREATE INDEX idx_fk_address_id ON public.customer USING btree (address_id)
public	customer	idx_fk_store_id	CREATE INDEX idx_fk_store_id ON public.customer USING btree (store_id)
public	customer	idx_last_name	CREATE INDEX idx_last_name ON public.customer USING btree (last_name)

Mevcut İndekslerin İncelenmesi

Başlangıç aşamasında zaten birçok indeksin tanımlı olduğu görülmektedir. Özellikle primary key ve foreign key alanlarında indeksler bulunmaktadır. Ayrıca film.title, actor.last_name ve customer.last_name gibi sorgularda sık kullanılabilecek alanlar için indeks tanımlamaları mevcuttur. Bu nedenle öncelikle optimizasyon için doğrudan indeksleri silmek yerine, önce hangi indekslerin gerçekten etkili olduğunun ölçülmesi planlanmaktadır.

Veritabanı izleme

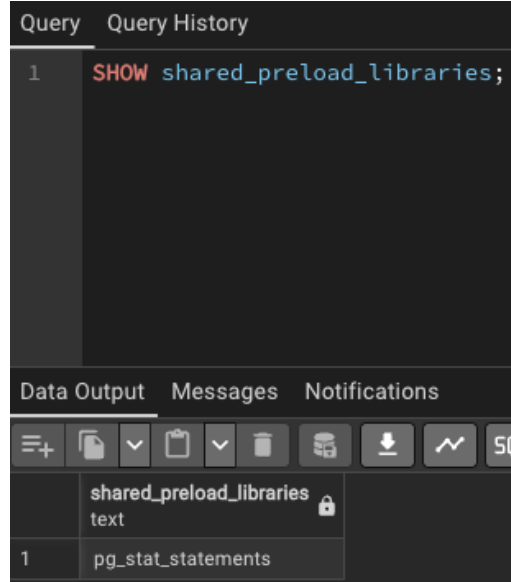
Bu bölümde pagila veritabanı için PostgreSQL’de bulunan **pg_stat_activity**, **pg_stat_statements** ve **log ayarları** modülü incelenmiştir. Bu araçlar sayesinde aktif sorgular, uzun süren işlemler ve performans problemi oluşturabilecek sorgular gözlemlenebilmektedir.

Modül Ayarları

Modüllerin çalışabilmesi için config dosyasında bazı değişiklikler yapmak gerekiyordu. postgresql.conf içindeki:

`shared_preload_libraries = ‘pg_stat_statements’,`

satırı varsayılan olarak boştu, onu `pg_stat_statements` ile değiştirdim. Bu adım, sorguların istatistiksel olarak izlenebilmesi için gereklidir.



SHOW Sorgusu İle Teyit

Ekleri Oluřturma

Yapılandırma ayarları tamamlandıktan sonra Pagila veritabanısında **pg_stat_statements** eklentisi oluşturulması gerekmektedir. Bu eklenti sayesinde veritabanısında çalışan sorguların çağrı sayısı, toplam çalışma süresi ve ortalama çalışma süresi gibi istatistikler elde edilebilmektedir.

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;  
SELECT * FROM pg_stat_statements LIMIT 5;
```

Sorgular

calls bigint	total_exec_time double precision	min_exec_time double precision	max_exec_time double precision	mean_exec_time double precision	stddev_exec_time double precision
5	0.44625	0.052375	0.138041	0.08925	0.035559982975249024
7	0.15487499999999998	0.011875	0.0375	0.022125000000000002	0.008901610127226582
7	6.663416	0.479125	1.4835	0.9519165714285714	0.4047192750236558
2	0.03229	0.007541	0.024749	0.016145	0.008604
5	0.029042	0.002666	0.00725	0.0058084	0.0016268111875691047

pg_stat_statements Sorgusundan Bir Parça

Yukarıdaki ekran görüntüsünde sorguların kaç defa çalıştığı, çalışma süreleri vs. görülebilmektedir.

Aktif Sorguları İzleme

PostgreSQL'de aktif oturumları ve çalışan sorguları izlemek için **pg_stat_activity** görünümü kullanılmıştır. Bu görünüm, o anda çalışan işlemleri, kullanıcı bilgisini, sorgu başlama zamanını ve bekleme durumlarını göstermektedir. Test amacıyla bir oturumda **SELECT pg_sleep(20);** sorgusu çalıştırılmış ve diğer oturumda **pg_stat_activity** ile bu sorgunun aktif olarak sistemde görüldüğü gösterilmiştir. Böylece anlık sorgu izleme işlemi gerçekleştirilmiştir.

Query

Query History

3

username,

4

datname,

5

state,

6

wait_event_type,

7

wait_event,

8

now() - query_start AS duration,

9

query

10

FROM pg_stat_activity

11

WHERE state <> 'idle'

12

ORDER BY duration DESC;

Data Output

Messages

Notifications

Aktif Sorguları Görme

Yukarıdaki tabloda diğer oturumda aktif olarak çalışan sleep sorgusunu, ne kadar zamandır çalıştığını vs. görebiliyoruz.

En Çok Zaman Harcayan Sorguları Tespit Etme

```
SELECT
    calls,
    round(total_exec_time::numeric, 2) AS total_ms,
    round(mean_exec_time::numeric, 2) AS mean_ms,
    rows,
    query
FROM pg_stat_statements
ORDER BY mean_exec_time DESC
LIMIT 10;
```

Ortalama Süreye Göre Sıralama

pg_stat_statements, sorguların çağrı sayısını, toplam planlama ve yürütme süresini, ortalama süreyi ve satır sayılarını gösterir.

	calls bigint	total_ms numeric	mean_ms numeric	rows bigint	query text
1	3	50003.45	16667.82	3	SELECT pg_sleep(\$1)
2	1	22.05	22.05	0	CREATE EXTENSION IF NOT E
3	3	40.68	13.56	2874	SELECT f.title, COUNT(r.r
4	3	36.17	12.06	1797	SELECT c.customer_id, c
5	1	2.51	2.51	0	ALTER SYSTEM SET log_min_
6	2	4.44	2.22	0	ALTER SYSTEM SET deadlock
7	1	1.50	1.50	0	ALTER SYSTEM SET log_lock_
8	5	7.24	1.45	5	SELECT set_config(\$1,\$2,\$3)
9	19	18.23	0.96	442	SELECT DISTINCT att.attname
10	1	0.87	0.87	1	SELECT set_config(\$1,\$2,\$3)

Sorguların Çalışma Zamanına Göre Sıralanmış Hali

Yavaş Sorguları Loglama

Yavaş süren sorguları kayıt altına almak için **log_min_duration_statement** parametresi kullanılmıştır. Test amaçlı bu değer 0 olarak belirlenmiştir ki tüm sorguları görebilelim fakat isteğe bağlı bu değer değiştirilerek beklenenden fazla uzun süren sorgular loglanabilir, tespiti halinde sorunun nerede olduğu görülebilir.

```

2026-04-05 15:01:26.441 +03 [28542] LOG: starting PostgreSQL 17.6 (Homebrew) on aarch64-apple-darwin24.4.0, compiled by Apple
clang version 17.0.0 (clang-1700.0.13.3), 64-bit
2026-04-05 15:01:26.442 +03 [28542] LOG: listening on IPv6 address "::1", port 5432
2026-04-05 15:01:26.442 +03 [28542] LOG: listening on IPv4 address "127.0.0.1", port 5432
2026-04-05 15:01:26.442 +03 [28542] LOG: listening on Unix socket "/tmp/.s.PGSQL.5432"
2026-04-05 15:01:26.445 +03 [28546] LOG: database system was shut down at 2026-04-05 15:01:25 +03
2026-04-05 15:01:26.449 +03 [28542] LOG: database system is ready to accept connections
2026-04-05 15:01:33.620 +03 [28574] LOG: duration: 4.047 ms statement: SET DateStyle=ISO; SET client_min_messages=notice;
SELECT set_config('bytea_output','hex',false) FROM pg_show_all_settings() WHERE name = 'bytea_output'; SET
client_encoding='utf-8';
2026-04-05 15:01:33.625 +03 [28574] LOG: duration: 0.296 ms statement: SELECT version()
2026-04-05 15:01:33.627 +03 [28574] LOG: duration: 2.535 ms statement:
SELECT
db.oid as did, db.datname, db.dataallowconn,
pg_encoding_to_char(db.encoding) AS serverencoding,
has_database_privilege(db.oid, 'CREATE') as cancreate,
datistemplate

```

Log Dosyasının Bir Kısımı

Yukarıda PostgreSQL servisinin başlatıldığını, Sunucunun bağlantıyı kabul etmeye hazırlandığını, her bir işlemin ne kadar sürdüğü görülebilmektedir. Burada minimum Log zamanı 0 yerine başka bir değer ayarlanırsa, filtreleme ile istenilen sonuç daha rahat görülebilir.

İndeks Yönetimi

Bu bölümde pgila veritabanı üzerinde inde işlemlerini inceleyeceğiz. Amaç, sorgu performansını artırmak için uygun indeksleri belirlemek, mevcut indeksleri görüntülemek ve yapılan indeksleme işleminin sorgu sürelerine etkisini ölçmektir. Bunları yapabilmek için **pg_indexes**, **pg_stat_user_indexes** ve **EXPLAIN ANALYZE** araçlarını kullanacağım.

Mevcut İndeksleri Görüntüleme

Burada mevcut olan indeksler **pg_indexes** görünümü ile incelenmiştir. Aşağıdaki ekran fotoğrafında tablolar üzerinde hangi indekslerin tanımlı olduğu ve bu indekslerin hangi sütunlar için oluşturulduğu görülebiliyor.

schemaname	tablename	indexname	indexdef
public	actor	actor_pkey_incl	CREATE UNIQUE INDEX actor_pkey_incl ON public.actor USING btree (actor_id) INCLUDE (first_name, last_name)
public	actor	idx_actor_last_name	CREATE INDEX idx_actor_last_name ON public.actor USING btree (last_name)
public	address	address_pkey	CREATE UNIQUE INDEX address_pkey ON public.address USING btree (address_id)
public	address	idx_fk_city_id	CREATE INDEX idx_fk_city_id ON public.address USING btree (city_id)
public	category	category_pkey	CREATE UNIQUE INDEX category_pkey ON public.category USING btree (category_id)
public	city	city_pkey	CREATE UNIQUE INDEX city_pkey ON public.city USING btree (city_id)
public	city	idx_fk_country_id	CREATE INDEX idx_fk_country_id ON public.city USING btree (country_id)
public	country	country_pkey	CREATE UNIQUE INDEX country_pkey ON public.country USING btree (country_id)
public	customer	customer_pkey	CREATE UNIQUE INDEX customer_pkey ON public.customer USING btree (customer_id)
public	customer	idx_fk_address_id	CREATE INDEX idx_fk_address_id ON public.customer USING btree (address_id)

Şu Anki İndekslerin Bir Kısımı

İndeks Kullanım İstatistikleri

Burada hangi indeksin gerçekten kullanıldığını göreceğiz. **pg_stat_user_indexes** görünümünde yer alan değerler, şu ana kadar çalıştırılan sorgularda kullanılan birikmiş indeks kullanım istatistiklerini gösterir.

```
SELECT
    relname AS table_name,
    indexrelname AS index_name,
    idx_scan,
    idx_tup_read,
    idx_tup_fetch
FROM pg_stat_user_indexes
ORDER BY idx_scan DESC, relname;
```

	table_name name	index_name name	idx_scan bigint	idx_tup_read bigint	idx_tup_fetch bigint
1	customer	customer_pkey	24	24	24
2	film	film_pkey	6	6	6
3	inventory	inventory_pkey	6	6	0
4	rental	idx_fk_inventory_id	6	6	0
5	actor	idx_actor_last_name	0	0	0
6	actor	actor_pkey_incl	0	0	0
7	address	idx_fk_city_id	0	0	0

Kullanılan ve Kullanılmayan İndeksler

Tabloda görülen bilgiler şu ana kadar kadarki sorgulardan elde edilmiştir. Burada görülen satırla şunları ifade etmektedir.

table_name: indeksin ait olduğu tablo

index_name: indeksin adı

idx_scan: o indeks kaç kez taranmış

idx_tup_read: indeks üzerinden kaç kayıt okunmuş

idx_tup_fetch: tablodan gerçekten kaç satır çekilmiş

Fakat burada bazı indexlerin 0 görünmesi, bu indexin gereksiz olduğu anlamına gelmez. Veritabanı kullanıldıkça buradaki sayılar artacaktır ve az sayıda kalan indekslerin belki gereksiz olabileceğine karar verilebilir.

İndeks Eklenecek Sorgu

```
EXPLAIN ANALYZE
SELECT *
FROM payment
WHERE customer_id = 1;
```

EXPLAIN ANALYZE sorgusu

İndeks etkisini ölçebilmek için önce payment tablosu üzerinde filtreleme yapan bir sorgu girilmiştir. Bu sayede mevcut durumda sorgunun tabloyu nasıl taradığı gözlemlenecektir.

QUERY PLAN
Append (cost=0.28..67.09 rows=30 width=26) (actual time=0.367..2.827 rows=32 loops=1)
-> Index Scan using idx_fk_payment_p2007_01_customer_id on payment_p2007_01 payment_1 (cost=0.28..8.31 rows=2 width=26) (actual time=0.366..0.367 rows=2)
Index Cond: (customer_id = 1)
-> Index Scan using idx_fk_payment_p2007_02_customer_id on payment_p2007_02 payment_2 (cost=0.28..8.37 rows=5 width=26) (actual time=0.387..0.389 rows=5)
Index Cond: (customer_id = 1)
-> Seq Scan on payment_p0000_default payment_8 (cost=0.00..12.65 rows=3 width=26) (actual time=0.135..0.538 rows=3 loops=1)
Filter: (customer_id = 1)
Rows Removed by Filter: 609
Planning Time: 11.333 ms
Execution Time: 3.128 ms

EXPLAIN ANALYZE Çıktısı

Burada tabloda gerekli yerlerde Index Scan gerekli yerlerde Seq Scan kullanıldığı görülmüştür. Toplam çalışma süresi ise 3.128 ms'dir.

Index Eklendikten Sonra Performans

Aynı sorguyu bir de indeks ekleyip deneyeceğiz. **customer_id** sütunu üzerinde bir indeks oluşturulmuştu. Bu sayede toplam çalışma süresinin ne ölçüde değiştiğine bakacağız

```
CREATE INDEX idx_payment_customer_id
ON payment (customer_id);
```

İndeks Ekleme Komutu

QUERY PLAN
Append (cost=0.28..62.77 rows=30 width=26) (actual time=0.010..0.126 rows=32 loops=1)
-> Index Scan using idx_fk_payment_p2007_01_customer_id on payment_p2007_01 payment_1 (cost=0.28..8.31 rows=2 width=26) (actual time=0.010..0.010 rows=2)
Index Cond: (customer_id = 1)
-> Index Scan using idx_fk_payment_p2007_02_customer_id on payment_p2007_02 payment_2 (cost=0.28..8.37 rows=5 width=26) (actual time=0.007..0.008 rows=5)
Index Cond: (customer_id = 1)
-> Index Scan using idx_fk_payment_p2007_03_customer_id on payment_p2007_03 payment_3 (cost=0.28..8.39 rows=6 width=26) (actual time=0.006..0.008 rows=6)
Index Cond: (customer_id = 1)
-> Index Scan using payment_p0000_default_customer_id_idx on payment_p0000_default payment_8 (cost=0.28..8.33 rows=3 width=26) (actual time=0.026..0.027 rows=3)
Index Cond: (customer_id = 1)
Planning Time: 2.077 ms
Execution Time: 0.175 ms

Yukarıda da görüldüğü gibi aynı sorgu çalıştırıldığında toplam çalışma süresi 0.175 ms'ye düşmüştür.

Ancak burada dikkat edilmesi gereken konu, her ne kadar çalışma süresi düşse de inden kullanımı fazladan diskte yer kaplayacaktır. Ayrıca çok düşük ve sıfır kullanım gösteren indexler kaldırılabilir fakat bu işlem yapılmadan önce indeksin uygulama açısından gerekli olup olmadığı analiz edilmelidir.

Sorgu İyileştirme

Bu bölümde sorgu iyileştirme işlemleri gerçekleştirilecektir. Amaç, uzun süren veya verimsiz çalışan bir sorguyu analiz etmek, neden yavaş çalıştığını incelemek ve aynı sonucu daha verimli üreten yeni bir sorgu yazmaktır. EXPLAIN ANALYZE ile bu işlemler gerçekleştirilecektir.

Sorgu Farkları

Burada aynı amaca hizmet eden iki farklı sorgu yazacağız ve ne kadar zamanda tamamlandıklarını analiz edeceğiz. Neden çalışma zamanlarının farklı olduğu tespit edilecektir.

```
EXPLAIN ANALYZE
SELECT
  c.customer_id,
  c.first_name,
  c.last_name,
  (
    SELECT SUM(p.amount)
    FROM payment p
    WHERE p.customer_id = c.customer_id
  ) AS total_payment
FROM customer c;
```

1.Sorgu

18	Planning Time: 0.299 ms
19	Execution Time: 9.689 ms

Çalışma Süresi

İlk aşamada müşteri bazlı toplam ödeme bilgisini getiren bir sorgu seçilmiştir. Bu sorguda her müşteri için ödeme toplamı ayrı bir alt sorgu ile hesaplanmaktadır. yapılan incelemede sorgunun çalışma zamanı incelenmiştir. Bu yapı, aynı işlemin çok sayıda tekrar yapılmasına neden olabildiği için sorgu performansı açısından incelenmeye uygundur.

```
EXPLAIN ANALYZE
SELECT
  c.customer_id,
  c.first_name,
  c.last_name,
  SUM(p.amount) AS total_payment
FROM customer c
LEFT JOIN payment p
  ON p.customer_id = c.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name;
```

2.Sorgu

25	Planning Time: 0.498 ms
26	Execution Time: 22.640 ms

Çalışma Süresi

Aynı sonucu daha verimli üretmek amacıyla sorgu yeniden düzenlenmiştir. İyileştirilmiş sürümde her müşteri için ayrı alt sorgu çalıştırmak yerine `payment` tablosu `LEFT JOIN` ile birleştirilmiş ve toplam ödeme değeri `SUM` ile tek sorgu içinde hesaplanmıştır. Böylece sorgunun daha az tekrar içeren ve daha bütüncül bir planla çalışması hedeflenmiştir.

Veri Yönetici Roller

Bu bölümde amaç, farklı roller için erişim yönetimini göstermektir. PostgreSQL'de bunun temeli **role** kavramıdır; bir role hem kullanıcı gibi davranabilir hem de grup gibi yetki taşıyabilir. Roller veritabanı nesnelere sahip olabilir ve bu nesneler üzerinde başka rollere yetki verebilir. Yetkiler `GRANT` ve `REVOKE` ile yönetilir; roller ayrıca grup mantığıyla başka rollere üye de yapılabilir.

```
CREATE ROLE pagila_admin LOGIN PASSWORD 'inanimazguclusifre123!admin';  
CREATE ROLE pagila_readonly LOGIN PASSWORD 'inanimazguclusifre123!readonly';  
CREATE ROLE pagila_dataentry LOGIN PASSWORD 'inanimazguclusifre123!dataentry';
```

3 Farklı Rol

Farklı erişim seviyelerinde 3 tane rol oluşturdum. **pagila_admin** yönetici rolü, **pagila_readonly** sadece okuma rolü ve **pagila_dataentry** veri giriş rolü olarak tanımlanmıştır. Bu roller sisteme kullanıcı gibi bağlanabilmektedir.

Erişim Yetkisi

Roller oluşturulduktan sonra ilgili kullanıcıların veritabanına bağlanabilmesi için CONNECT yetkisi verilmiştir. Ayrıca tabloların bulunduğu **public** şeması üzerinde USAGE yetkisi tanımlanmıştır.

```
GRANT CONNECT ON DATABASE pagila TO pagila_readonly, pagila_dataentry, pagila_admin;  
GRANT USAGE ON SCHEMA public TO pagila_readonly, pagila_dataentry, pagila_admin;
```

Erişim Yetkisi

Tablo Bazında Yetki

pagila_readonly rolüne **customer** ve **payment** tabloları üzerinde yalnızca **SELECT** yetkisi verilmiştir. **pagila_dataentry** rolüne bu tablolar üzerinde **SELECT**, **INSERT** ve **UPDATE** yetkileri tanımlanmıştır. **pagila_admin** rolüne ise aynı tablolar üzerinde tüm yetkiler verilmiştir. Bu yapı sayesinde kullanıcıların yalnızca ihtiyaç duydukları işlemleri yapabilmesi sağlanmıştır.

```
GRANT SELECT ON customer TO pagila_readonly;  
GRANT SELECT ON payment TO pagila_readonly;  
  
GRANT SELECT, INSERT, UPDATE ON customer TO pagila_dataentry;  
GRANT SELECT, INSERT, UPDATE ON payment TO pagila_dataentry;  
  
GRANT ALL PRIVILEGES ON customer TO pagila_admin;  
GRANT ALL PRIVILEGES ON payment TO pagila_admin;
```

Verilen Yetkileri Listeleme

Verilen yetkiler, **information_schema.role_table_grants** görünümü kullanılarak görülebilmektedir. Bu şekilde hangi role, hangi tablo üzerinde hangi yetkinin verildiği açık biçimde gözlemlenebilir.

```
SELECT
    grantee,
    table_schema,
    table_name,
    privilege_type
FROM information_schema.role_table_grants
WHERE table_name IN ('customer', 'payment')
ORDER BY grantee, table_name, privilege_type;
```

Yetki Listesini Görme Sorgusu

	grantee name	table_schema name	table_name name	privilege_type character varying
1	pagila_admin	public	customer	DELETE
2	pagila_admin	public	customer	INSERT
3	pagila_admin	public	customer	REFERENCES
4	pagila_admin	public	customer	SELECT
5	pagila_admin	public	customer	TRIGGER

18	pagila_dataentry	public	payment	INSERT
19	pagila_dataentry	public	payment	SELECT
20	pagila_dataentry	public	payment	UPDATE
21	pagila_readonly	public	customer	SELECT
22	pagila_readonly	public	payment	SELECT

Verilen Yetkiler

Verilen Yetkilerin Geri Alınması

Yalnızca yetki verme değil, aynı zamanda yetki alma işlemleri de yapılabilmektedir. Örnek olarak **pagila_dataentry** rolünün **payment** tablosu üzerindeki **UPDATE** yetkisi geri alınmıştır. **GRANT** ve **REVOKE** komutları ile yetkiler düzenlenebilmektedir.

```
REVOKE UPDATE ON payment FROM pagila_dataentry;
```

Yetki Alma

Veritabanı Yedekleme ve Felaketten Kurtarma Planı

(Proje - 2)

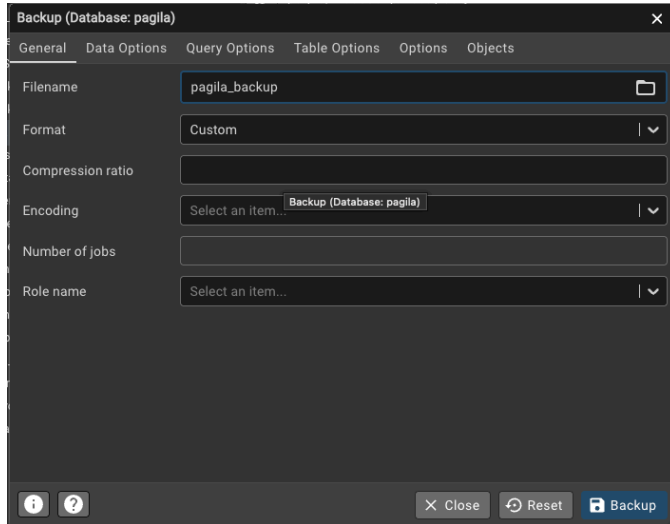
Bu projede örnek veritabanı kullanarak PostgreSQL ile yedekleme ve felaketten kurtarma planları yapılmıştır. Çalışmanın amacı; verilerin kaybolması veya bozulması durumunda, veritabanını güvenli bir şekilde geri döndürebilmek gibi konular içermektedir.

Tam, Artık, Fark Yedeklemeleri

Başlıkta görülen MSSQL kavramları PostgreSQL'de birebir aynı isimde sunulmasa da benzer ihtiyaçları karşılayan yedekleme mekanizmaları bulunmaktadır. Bu bölümde pgAdmin kullanılarak tam yedekleme gerçekleştirilecek, artık ve fark yedekleme için ise PostgreSQL'in incremental backup ve WAL tabanlı sistemi üzerinden gerçekleştirilecektir.

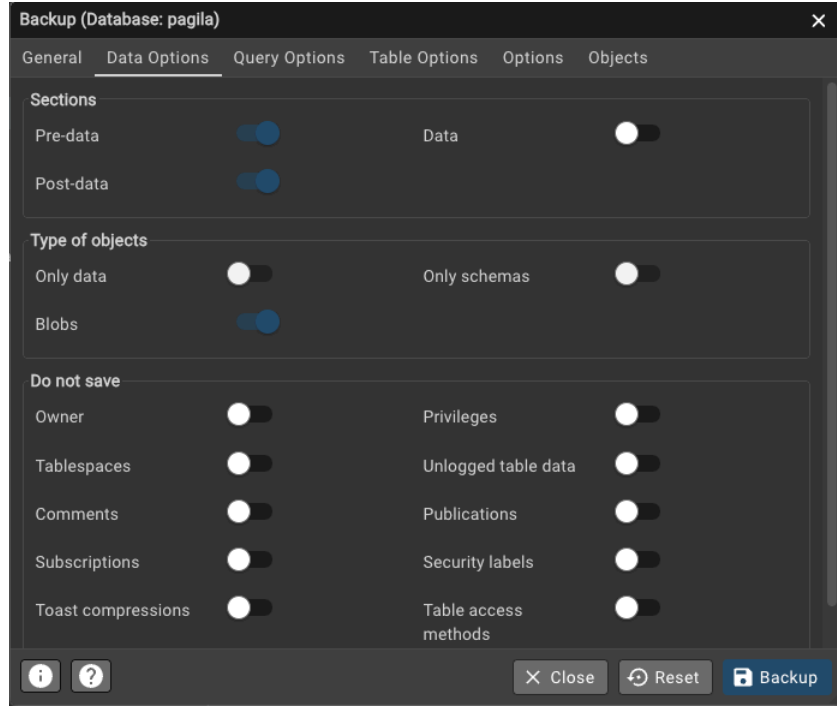
Tam Yedekleme

Veritabanının belirli bir zamandaki tüm yapısının tek parça halinde yedeklenmesi anlamına gelmektedir. Bu aşama pgAdmin üzerinden gerçekleştirilecektir.



General ekranında dosyanın adı, formatı, sıkıştırma oranı, encoding, paralel iş sayısı ve gerekirse rol adı burada seçilir. Format olarak 'Custom' seçilmesi **pg_store** ile geri yüklenebilmesidir.

General Ekranı



Data Options Ekranı

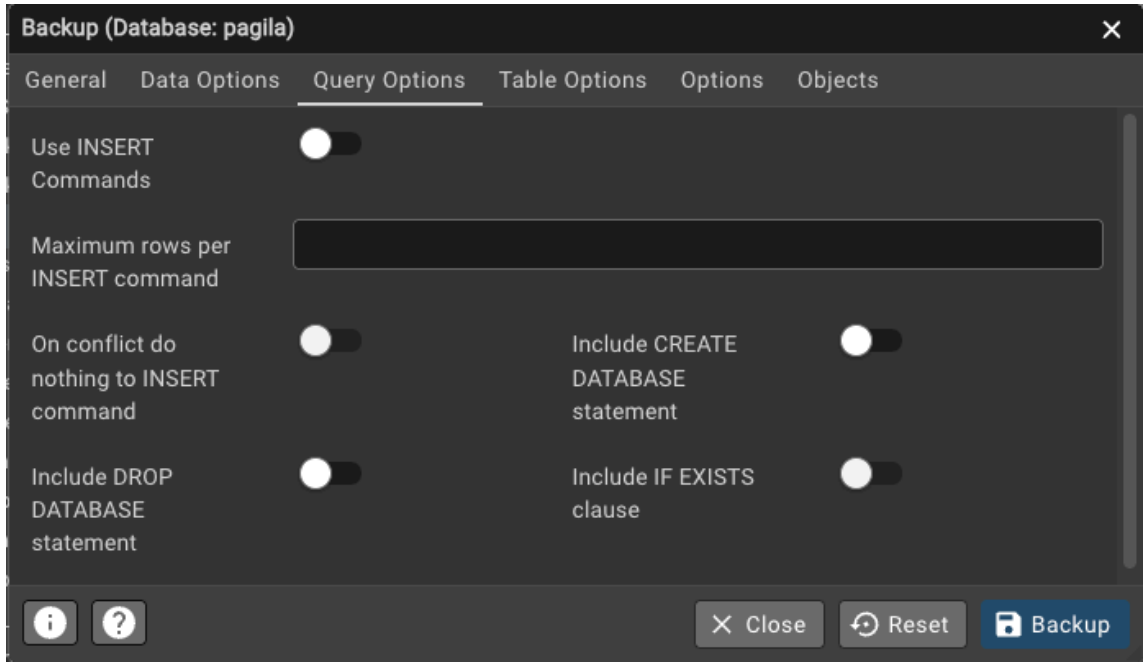
Bu sekme, yedeğin hangi bölümünü alacağını belirler. Burada üç ana bölüm var:

- **Pre-data:** tablo tanımları gibi veri öncesi şema öğeleri
- **Data:** gerçek tablo verileri, large objects ve sequence değerleri
- **Post-data:** index, trigger, rule ve constraint gibi sonradan kurulan öğeler.

‘Sections’ bölümünün altında ‘Type of object’ bölümü bulunmaktadır:

- **Only data** açılırsa sadece veri alınır
- **Only schemas** açılırsa sadece şema nesneleri alınır
- **Blobs** kapatılırsa large object’ler dışarıda bırakılır.

Son olarak ‘Do not save’ kısmında bazı metadata öğelerini yedekten hariç tutabiliyoruz. Bu kısım daha çok özel senaryolar için geçerlidir.



Query Options Ekranı

Bu sekme, verinin yedek dosyasında nasıl yazılacağını etkiler. Örneğin veri **COPY** yerine **INSERT** komutlarıyla yazılsın mı, her **INSERT** içine en fazla kaç satır girsin mi, **CREATE DATABASE** komutu eklensin mi, gerekirse önce **DROP DATABASE** çalışsın mı gibi ayarlar burada bulunur.

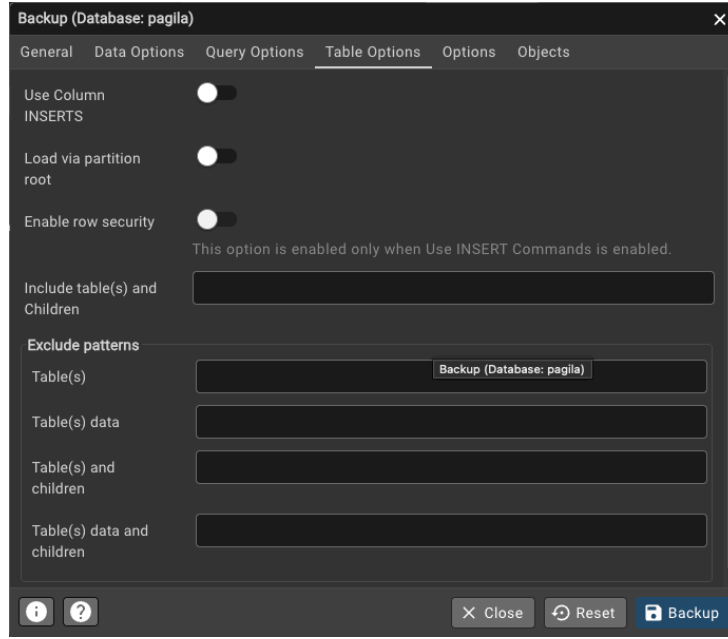
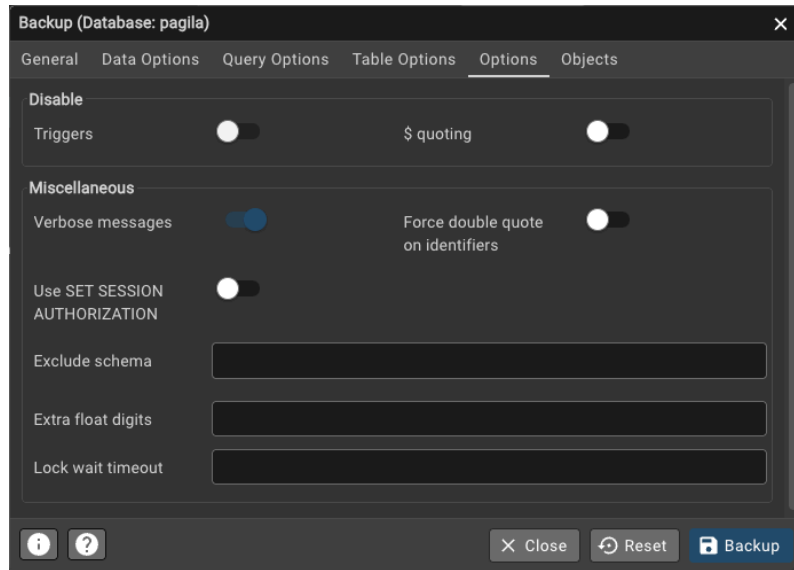


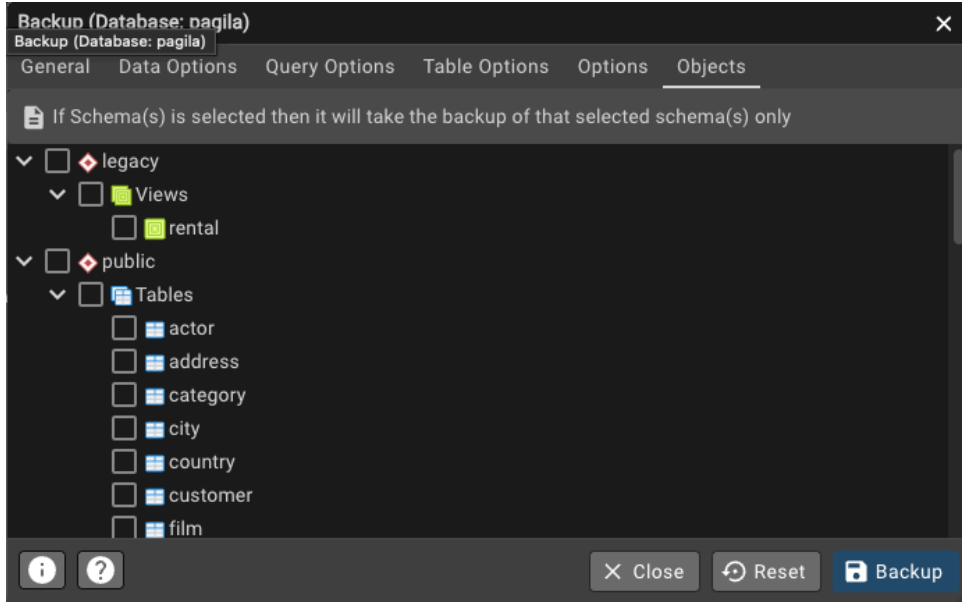
Table Options Ekranı

Burada tüm ayarları varsayılan olarak bırakmak bizim senaryomuzda mantıklı olacaktır. Satır güvenliği, belirli tabloları hariç tutmak vs. İstiyorsak bu ayarlarla oynayabiliriz.



Options Ekranı

Bu sekmede daha detaylı ayarlar bulunmaktadır. Örneğin, data-only backup sırasında restore esnasında trigger'ları geçici kapatma, belirli şemaları hariç tutma, float hassasiyeti gibi seçenekler mevcuttur. Bir öncekinde olduğu gibi bura da varsayıla bırakılabilir.



Object Ekranı

Yalnızca seçili nesnelerin yedeğini almak için kullanılır. Hangi şema seçilirse yalnızca onun yedeğini alır. Biz veritabanının tamamının yedeğini almak istediğimizden bu sekmeye dokunmuyoruz.

Artık ve Fark Yedekleme

Artık ve fark yedekleme yapmak için PostgreSQL'de bir tuş vs. yoktur. Bunun yerine fiziksel yedekleme aracı olan `pg_basebackup` komutu ile gerçekleştireceğiz. Önce terminal üzerinden full base backup alıp sonra `--incremental` parametresi ekleyerek bir önceki yedek ile şu anki durumun farkını yedekleyebiliriz. Bunları yapabilmek için de WAL özetleme özelliğinin açık olması gerekmektedir. Bunu `postgres.conf` dosyasını içinden aktif hale getirdim.

WAL hangi blokların değiştiğini takip edebilmesi için gereklidir. Bu yüzden ilk yedeklemeyi yapmadan önce WAL özetlemeyi açmak gerekir

```
macbook@semihs-MacBook-Pro ~ % pg_basebackup -U macbook -D ~/Desktop/pg_backups/
full_base -Fp
```

İlk Yedek İçin Gereken Komut

Yukarıdaki komut sayesinde ilk yedeğimizi alıyoruz. pg_basebackup, çalışan cluster'ın fiziksel base backup'ını alır. Bu yedek, ileride incremental backup ve kurtarma için başlangıç noktasıdır. Klasör içindeki backup manifest sayesinde –incremental ile fark yedeği alabileceğiz.

Yedek aldıktan sonra veritabanında ufak değişiklik yapıyoruz ki, ikinci yedekte sadece fark olan dosyaları yedekleyelim.

```
CREATE TABLE IF NOT EXISTS backup_test (
  id serial PRIMARY KEY,
  note text,
  created_at timestamp default now()
);

INSERT INTO backup_test (note)
VALUES ('İlk degisiklik - artik yedek oncesi');
```

Yapılan Küçük Değişiklik

Daha sonra –incremental ile ikinci yedeğimizi alıyoruz.

```
macbook@semihs-MacBook-Pro ~ % pg_basebackup -U macbook \
-D ~/Desktop/pg_backups/inc1 \
--incremental=/Users/macbook/Desktop/pg_backups/full_base/backup_manifest
```

İkinci Yedek

Belirlenen dosyaya ilk yedek ile farklı olan verileri yedekleyebiliyoruz.

Zamanlayıcılarla Yedekleme

Bu bölümde kullanıcı müdahalesi olmadan belli aralıklarla yedekleme yapılması amaçlanmıştır. MacOS ortamında çalıştığım için zamanlanmış görevler için **launchd** kullandım.

Yedekleme işlemi için PostgreSQL'in resmi yardımcı aracı olan **pg_dump** kullanılmıştır. **pg_dump**, çalışan bir PostgreSQL veritabanının mantıksal yedeğini alabilir ve veritabanı eşzamanlı olarak kullanılırken de tutarlı bir çıktı üretebilir.

Yedekleme Aşamaları

İlk olarak çalışabilmesi için plist dosyası tanımlamak gerekiyor. Bunu da aşağıdaki komut ile yapabiliyoruz.

```
macbook@seihns-MacBook-Pro ~ % plutil -lint ~/Library/LaunchAgents/com.seih.pagila.backup.plist
```

Plist Komutu

```
<?xml version="1.0" encoding="UTF-8"?>
<plist version="1.0">
<dict>
  <key>Label</key>
  <string>com.seih.pagila.backup</string>

  <key>ProgramArguments</key>
  <array>
    <string>/bin/bash</string>
    <string>-lc</string>
    <string>/opt/homebrew/opt/postgresql@17/bin/pg_dump -h localhost -U macbook -d pagila -F c -f
    "$HOME/Desktop/auto_backups/pagila_$(date +%Y%m%d_%H%M%S).backup"</string>
  </array>

  <key>StartInterval</key>
  <integer>60</integer>

  <key>RunAtLoad</key>
  <true/>

  <key>StandardOutPath</key>
  <string>/Users/macbook/Desktop/auto_backups/launchd.log</string>

  <key>StandardErrorPath</key>
  <string>/Users/macbook/Desktop/auto_backups/launchd.log</string>
</dict>
</plist>
```

Otomatik Yapıyı Oluşturan XML

Bir önceki sayfada komut olarak pg_dump tanımlı ve test için her 60 saniyede bir çalışacak şekilde ayarlanmıştır. Ancak gerçek senaryolar düşünüldüğünde bu süre 1 gün olarak ayarlanabilir. Sonra bu oluşturduğumuz plist dosyası aşağıdaki komut ile sisteme yüklenmiştir.

```
macbook@semihs-MacBook-Pro ~ % launchctl load ~/Library/LaunchAgents/com.semih.pagila.backup.plist
```

Felaketten Kurtarma Senaryoları

Bu adımda oluşabilecek veri kayıp durumlarında yedekten nasıl geri yüklenebileceğini anlatacağım. Daha önce pgAdmin üzerinden aldığımız backup dosyasını kullanarak geri yükleme aşamasını uygulayacağız.

Uygulama sırasında mevcut çalışma veritabanını riske atmamak için ayrı bir test veritabanı oluşturup geri yükleme işlemini bu test ortamında gerçekleştireceğim. Daha sonra veri kaybı senaryosu oluşturmak amacıyla belirli kayıtları silip, ardından aynı backup dosyasını kullanılarak restore işlemini uygulayacağım. Böylece yedeklerin felaket anında kullanılabilir olduğu uygulamalı olarak görmüş olacağız.

Restore Aşamaları

```
DELETE FROM film_actor
WHERE actor_id IN (1,2,3);

DELETE FROM actor
WHERE actor_id IN (1,2,3);
```

Silme Sorgusu

Yukarıdaki sorguyu kullanarak kontrollü olarak veri siliyoruz. Restore aşamasından sonra sildiğimiz bu verilerin geri geldiğini göreceğiz.

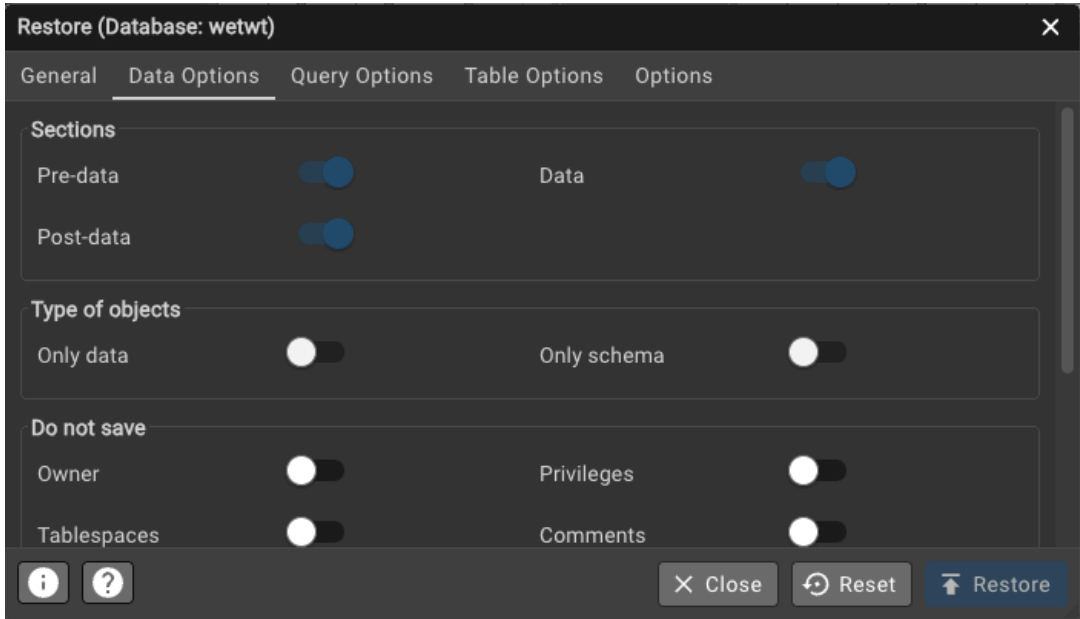
	actor_id [PK] integer	first_name character varying (45)	last_name character varying (45)	last_update timestamp without time zone
1	1	PENELOPE	GUINNESS	2006-02-15 09:34:33
2	2	NICK	WAHLBERG	2006-02-15 09:34:33
3	3	ED	CHASE	2006-02-15 09:34:33
4	4	JENNIFER	DAVIS	2006-02-15 09:34:33
5	5	JOHNNY	LOLLOBRIGIDA	2006-02-15 09:34:33

Veri Silinmeden Önce Tablo

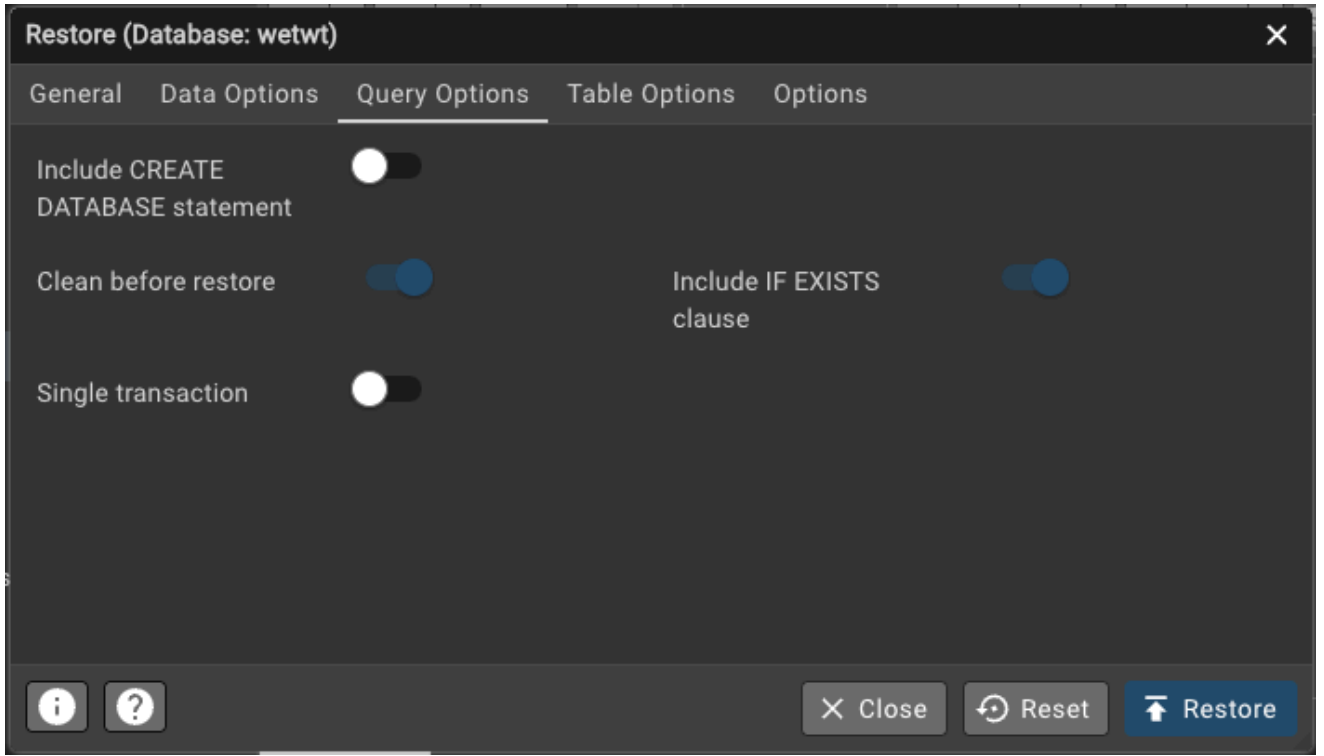
	actor_id [PK] integer	first_name character varying (45)	last_name character varying (45)	last_update timestamp without time zone
1	4	JENNIFER	DAVIS	2006-02-15 09:34:33
2	5	JOHNNY	LOLLOBRIGIDA	2006-02-15 09:34:33
3	6	BETTE	NICHOLSON	2006-02-15 09:34:33
4	7	GRACE	MOSTEL	2006-02-15 09:34:33
5	8	MATTHEW	JOHANSSON	2006-02-15 09:34:33

Veri Silindikten Sonra Tablo

Tüm verileri geri getirmek için aşağıda görülen Section içindekilerini açmak gerekmektedir. Ama sadece data sildiysek ve onu geri getirmek istiyorsak only data seçeneği seçebiliriz, şemayı yeniden kurması gerekmez.



Restore - Data Options Aşaması



Restore - Data Options Aşaması

Burada restore yapmadan önce çakışma olmaması için temizlik yapıyoruz. Çünkü tablolar yeniden oluşacak, temizlik yapmazsak bu tablo zaten var hatası verir. Include IF EXIST clause seçeneği Clean before restore ile birlikte kullanılır. DROP komutlarını DROP ... IF EXISTS şeklinde daha güvenli hale getirir. Böylece silinmeye çalışılan nesne yoksa gereksiz hata üretmez.

Test Yedekleme Senaryoları

pgAdmin üzerinden yedek alma işlemini tamamlamıştık. Daha sonra yine pgAdmin üzerinden kaybolan verilerimiz için restore işlemini yaptık. Yani bu senaryoda aldığımız yedeklerin doğruluğunu test etmiş olduk.

